Guide Pup: Vision Assistant

One-page repo summary based on README, Expo config, routes, AI logic, and settings code.

What It Is

- Voice-first Expo / React Native prototype for camera-based scene guidance for visually impaired users.
- Current repo state: accessible navigation flow plus a second, more stylized capture / inspiration flow; production backend is not found in repo.

Who It's For

- Primary persona: blind or low-vision users who need spoken scene descriptions and simple movement cues while holding a phone.

What It Does

- Onboarding explains permissions and voice-first usage.
- Home screen starts guidance with one large touch target; long press SOS is a placeholder.
- Navigation mode captures frames about every 4.5s and speaks directions plus scene context.
- Vision analysis returns obstacles, path clearance, hazard level, lighting, surface type, and a recommended direction.
- Haptics reinforce stop / turn / forward states.
- Settings persist speech rate, description detail, and a bounding-box debug toggle in AsyncStorage.
- A separate tabbed capture screen supports one-shot analysis; an inspiration tab shows static creative cards and ambient audio.

How It Works

- Expo Router app is wrapped by QueryClientProvider, SettingsProvider, and VoiceProvider.
- CameraView captures base64 frames in navigation and capture screens.
- GuideAI calls VisionAI; VisionAI uses @rork-ai/toolkit-sdk generateObject with a Zod schema to structure analysis.
- GuideAI smooths the result, then the UI speaks it with expo-speech and reinforces it with expo-haptics.
- Local settings are stored in AsyncStorage.
- Backend / auth / database / server code: Not found in repo.

How It Is Now

- Best described as a polished prototype, not a launch-ready mobility product.
- Strongest path today is the simple accessibility flow (`/`, `/navigation`, `/settings`, `/onboarding`).
- Product identity is split: the tabbed capture / inspiration experience reads more like a creative camera concept than assistive navigation.

How To Enhance

- Short term: keep Expo and tighten the mobility use case before adding more surfaces.
- SwiftUI is worth it only if the launch target becomes iOS-first and you need lower-latency camera / audio loops, deeper VoiceOver polish, AVFoundation / Vision hooks, or background behavior that Expo cannot cover cleanly.
- If cross-platform remains the goal, add native modules only for the bottlenecks instead of rewriting the full app first.

Launch + Real Backend

- Move vision calls behind a real backend so API keys are never shipped in-app; add rate limiting, logging, prompt / model versioning, and failure monitoring.
- Replace placeholder SOS and mock sensor starters with real services and explicit safety behavior.
- Resolve the route split: ship either the accessibility navigator or fold the tabbed capture UI into the same product story.
- Add QA on real devices, analytics / crash reporting, privacy copy, and EAS release setup.
- User accounts, persistence, offline strategy, and human-support workflows: Not found in repo.

How To Run

- cd expo
- bun install
- Create .env with EXPO_PUBLIC_OPENAI_API_KEY and optional EXPO_PUBLIC_OPENAI_MODEL
- bun run start for device / simulator, or bun run start-web for browser
- For store builds, README points to eas build --platform ios / android
- Note: package.json start scripts call bunx rork start ...; a plain expo start script is not found in repo.

Evidence: expo/README.md, expo/app.json, expo/package.json, expo/app/_layout.tsx, expo/src/logic/VisionAI.ts, expo/src/logic/GuideAI.ts, expo/src/screens/{HomeScreen,NavigationScreen,OnboardingScreen,SettingsScreen}.tsx, expo/app/(tabs)/{capture,inspiration}.tsx, expo/src/providers/SettingsProvider.tsx.